POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
Overfit & Chill

Andrea Ardu, Lorenzo Cattaneo, Simone Controguerra, Alberto Furlan

andreaaardu, loreo21, simonecontroguerra, furlanalberto

960988, 956356, 258927, 286758

November 17, 2025

1 Introduction

The objective of this project is to train a **neural network** using multivariate **time series samples** and use it to **classify** subjects according to their pain level. Each sample corresponds to a *registration period* for one subject and contains features describing *pain*, *physical condition*, and *motion-related joint measurements*. The goal is to build a model that correctly predicts one of the three labels: *no_pain*, *low_pain*, or *high_pain*.

2 Problem Analysis

2.1 Dataset Analysis and Preprocessing

To develop a robust model, we first performed an extensive analysis of the dataset. Our investigation focused on:

- **Outlier detection**:
  Conducted using the IQR method to identify extreme values in joint trajectories.
- **Autocorrelation analysis (ACF/PACF)**:
  Used to characterize temporal dependencies and detect recurring local patterns. The ACF highlighted slowly decaying correlations, suggesting non-random structure, while the PACF revealed short-range dependencies across consecutive time steps. These observations motivated a **window-based approach** to capture localized temporal dynamics.

Based on this analysis, we performed these steps:

- Removal of **zero-variance** and highly correlated features.
- Cyclical **time encoding** using sinusoidal and positional indicators.
- Derivation of **velocity** and **acceleration** features to enrich temporal information.
- Window extraction guided by **autocorrelation-driven** window size selection.

Each sample contains 30+ dynamic and static features, including:

- *sample_index*: Sample identifier.
- *time*: Time step.
- *pain_survey*: Four pain descriptors.
- *n_legs/hands/eyes*: Static categorical body-condition features.
- *joint*: Multivariate joint angle measurements.

The dataset presents challenges such as class imbalance and heterogeneity between subjects due to missing limbs. These issues guided both our preprocessing and modeling strategy.

3 Method

We combined several methodological components to create an effective learning pipeline:

3.1 Preprocessing

- Removal of **low-variance signals** and highly correlated *joints*.
- Increment of *survey* values to avoid zeros in mathematical operations.
- **Standardization** of dynamic features.
- Creation of **velocity** and **acceleration** channels.
- **Embedding** of static categorical features for richer representation.
- **Sine and cosine** useful for periodic, cycling data and temporal series

We first adopted **stratified split** to obtain the Holdout test-set, ensuring each subject appeared in only one split while preserving label distribution. Then for the model training [**?**]. Time series were segmented into windows (`window size = autocorrelation-based`, `stride = 25%`) to capture local temporal structures. To address class imbalance, we adopted **focal loss**, **effective-number weighting**, **label smoothing** and **Oversampling**.

3.2 Model Architecture

The final model consists of:

- **Multi-layer CNN encoder**: captures local temporal patterns in joint trajectories.
- **BiLSTM encoder**: models long-range temporal dependencies.
- **Additive attention mechanism**: aggregates temporal information and improves interpretability.
- **Static feature embedding**: encodes subject metadata (e.g. patients with anomalies).
- **Feed-forward MLP head**: produces the final class prediction, Linear Layers, RELU activations and Dropout.

Additional improvements include gradient clipping for training stability and address possible overfitting

4 Experiments

We experimented with several models to obtain the final model described above.

4.1 WinBiGru-Baseline

The *WinBiGru* baseline consists of a bidirectional GRU (`BiGRU`) encoder followed by a simple classifier head. Instead of feeding entire subject trajectories to the network, the raw time series data are segmented into fixed-length overlapping windows. Each window is treated as an independent sequence and encoded through the BiGRU, which captures temporal dependencies in both directions.
The final hidden representation of each window is then mapped to a pain class by a fully connected classifier.

4.2 CNNBiLSTM-Baseline

The *CNNBiLSTM* baseline augments the recurrent architecture with a convolutional front-end.
A 1D CNN is first applied along the temporal dimension to each multivariate sequence window. This convolutional stage acts as a local feature extractor, extracting short-range temporal patterns and attenuating high-frequency sensor noise. The resulting feature maps are then fed into a bidirectional LSTM (`BiLSTM`) that models longer-range dependencies and contextual information in both temporal directions.

4.3 CNN-BiLSTM-Focalloss-Ensembling-Baseline

This baseline builds on the CNN+BiLSTM architecture and introduces two key improvements: Focal Loss for training and model ensembling at inference time. Focal Loss replaces standard cross-entropy to better handle the strong class imbalance between `no_pain`, `low_pain`, and `high_pain`. During training, it down-weights easy, majority-class examples and focuses the updates on hard and minority-class samples. At inference, instead of relying on a single model, we train multiple models on different cross-validation folds and ensemble their predictions by averaging class probabilities.

4.4 CNN-BiLSTM-FocalLoss-SMOTE-Baseline

This baseline uses the same *CNN+BiLSTM* backbone and Focal Loss, but additionally introduces SMOTE (Synthetic Minority Over-sampling Technique) at the window level to address class imbalance in the training set. Before training each fold, we apply SMOTE to oversample `low_pain` and `high_pain` windows in feature space while leaving the majority `no_pain` class unchanged. SMOTE generates synthetic windows by interpolating between existing minority samples, increasing their effective count without simply duplicating data.

5 Discussion

We trained a **windowed CNN-BiGRU** with **additive attention** and mean**-probability aggregation**, augmented with **velocity features** and light time masking. A **hybrid loss** with effective-number class weights and a balanced window sampler improved **F1**, by increasing **minority-class recall**, while **attention entropy regularization** reduce overfitting. **Static categorical embeddings** added a small and consistent gain. Most residual errors are confusions between *low* and *high pain*, especially on subject types underrepresented in training dataset. The confusion matrix remains strongest on *no_pain*. Limitations include restricted **hyperparameter** search.

Table 1: F1 scores according to the submissions' results

Model	**Val**	**Test**
Win-BiGRU Baseline	82%	79%
CNN-BiLSTM Baseline	90.8%	87.9%
CNN-BiLSTM + Focal Loss + Ensembling	96.52	94.48%
CNN-BiLSTM + SMOTE	**98.7%**	**94.89%**

6 Conclusions

The proposed CNN–BiLSTM architecture with attention, patient-class embedding, and autocorrelation-guided windowing achieves strong performance on the Pirate Pain task while avoiding subject-level data leakage. A 1D convolutional front-end plus bidirectional LSTM and multi-head self-attention captures both local motion patterns and longer-range temporal dependencies, and the patient-class embedding injects informative static information. On the training side, window-level SMOTE synthesizes additional low-pain and high-pain examples, while focal loss with label smoothing and data-driven class weights rebalances optimization toward hard and minority cases. Stratified group cross-validation at the subject level prevents leakage and yields an average macro F1 of 0.8861 $\pm$ 0.0464. On the held-out test split, the best single fold reaches a macro F1 of 0.8411, with high accuracy for no pain and competitive performance on low and high pain. Overall, SMOTE plus focal loss improves minority-class recognition over simpler baselines, and the CNN–BiLSTM–attention backbone offers a strong, leakage-free baseline for future extensions.

7 Individual Contributions

Each team member took part in almost every aspect of the project, however this was the main work distribution:

- Simone Controguerra: took part in data analysis.
- Lorenzo Cattaneo: took part in hyper parameter tuning.
- Andrea Ardu: took part in network creation and data analysis.
- Alberto Furlan: took part in network creation and hyper parameter tuning.